

Python Avancado

Programacao Assincrona, Concorrencia, TDD e Algoritmos

Concorrencia

async / threads / processos

Testes

TDD com pytest

Algoritmos

Big O / Binary / Bubble

Projeto

Web scraping assincrono

SUMARIO

Glossario de termos essenciais

Aula 1 — Sincrono, assincrono e processos

Aula 2 — Yield e Generators

Aula 3 — Coroutines e o event loop

Aula 4 — CPU parte 1: multiprocessing

Aula 5 — Threads parte 2

Aula 6 — TDD (Test Driven Development)

Aula 7 — Binary Search

Aula 8 — Bubble Sort

Aula 9 — Ferramentas, sites e comunidades

Exercicio final — Web scraping assincrono

Parte A: o codigo completo

Parte B: explicacao linha por linha

REFERENCIA

Glossario de termos essenciais

Este modulo usa muitos termos que se confundem facilmente. Leia este glossario antes das aulas e volte a ele sempre que precisar. Cada termo e retomado em profundidade no capitulo correspondente.

Concorrencia e paralelismo

Termo	O que e	Analogia
Sincrono	Uma tarefa por vez; cada uma espera a anterior terminar.	Fila unica no caixa
Assincrono	Enquanto uma tarefa espera I/O, outra roda; nao bloqueia.	Garcom que anota varias mesas sem esperar a cozinha
Concorrencia	Varias tarefas em progresso, alternando o uso do recurso.	Um cozinheiro alternando entre panelas
Paralelismo	Varias tarefas rodando de fato ao mesmo tempo.	Varios cozinheiros, um por panela

Unidades de execucao

Termo	Definicao
Processo	Instancia de um programa, com memoria propria e isolada.
Thread	Unidade de execucao dentro de um processo; compartilha a memoria dele.
Coroutine	Funcao que pode pausar (await) e retomar, gerenciada pelo event loop.
Event loop	Agendador que decide qual coroutine roda a cada ponto de pausa.
Task	Coroutine ja agendada no event loop, rodando concorrentemente.

Conceitos-chave

Termo	Definicao
GIL	Global Interpreter Lock: trava do CPython que so deixa UMA thread executar bytecode Python por vez.
I/O-bound	Tarefa limitada por espera externa (rede, disco, banco).
CPU-bound	Tarefa limitada por processamento (calculo, hashing).
Generator	Funcao com yield que produz valores sob demanda, guardando o estado.
yield	Pausa a funcao, devolve um valor e preserva o ponto de execucao.
await	Suspende a coroutine atual e devolve o controle ao event loop.
Semaphore	Contador que limita quantas tarefas entram numa secao ao mesmo tempo.
Lock	Garante que so uma thread execute a secao critica por vez.
Race condition	Bug por acesso concorrente nao sincronizado ao mesmo dado.
TDD	Test Driven Development: escreve o teste antes do codigo.
Big O	Notacao que descreve como o custo cresce com o tamanho da entrada.
IPC	Inter-Process Communication: troca de dados entre processos isolados.

Nota

Regra mental para o modulo inteiro: I/O-bound pede assincrono/threads; CPU-bound pede multiprocessing. Quase toda decisao de concorrencia em Python nasce dessa distincao por causa do GIL.

AULA 1

Sincrono, assincrono e processos

Codigo sincrono

Em codigo sincrono, cada instrucao espera a anterior terminar. A thread principal fica bloqueada ate a operacao concluir. Se a consulta ao banco leva 200ms, a aplicacao trava por 200ms naquela linha.

```
def get_product(product_id):  
    product = Product.objects.get(id=product_id)  
    return product
```

Codigo assincrono

Assincrono libera a thread para outra tarefa enquanto espera. O await nao e uma espera bloqueante: e um ponto onde a funcao devolve o controle ao event loop, que pode rodar outra coroutine enquanto o banco nao responde.

```
import asyncio  
  
async def get_product(product_id):  
    product = await Product.objects.aget(id=product_id)  
    return product  
  
async def main():  
    product = await get_product(1)  
    print(product)  
  
asyncio.run(main())
```

Nota

Voce ja conhece esse modelo do Node.js: o event loop do JavaScript funciona no mesmo principio (single-thread, nao bloqueante). A diferenca e que em Python isso e opt-in: voce declara async def e liga o loop com asyncio.run().

Processo, thread e o GIL

Um processo e uma instancia de um programa. As unidades se relacionam assim:

Conceito	O que e	Isolamento
Processo	Instancia com memoria propria	Total
Thread	Execucao dentro de um processo	Compartilha memoria

O GIL (Global Interpreter Lock) muda tudo em Python: o CPython so permite que uma thread execute bytecode Python por vez, mesmo em maquina com varios nucleos. Consequencia pratica:

- Threads NAO dao paralelismo real para codigo Python puro (CPU-bound). Ajudam em I/O, porque a thread libera o GIL enquanto espera.
- Para paralelismo real de CPU, use multiplos processos (multiprocessing): cada processo tem seu proprio interpretador e seu proprio GIL.

Guia de decisao

Cenario	Ferramenta
Muitas requisicoes HTTP / consultas ao banco / I/O de rede	asyncio
I/O legado, sem suporte a async	threading
Processamento pesado de CPU (calcula, hashing, ML)	multiprocessing

Atencao

Na otica de seguranca: um endpoint sincrono que bloqueia o event loop de uma API async trava o servico inteiro sob carga. E um vetor comum de DoS acidental — sem ataque sofisticado nenhum.

Exercicio para fazer a mao

1. Explique com suas palavras a diferenca entre bloqueante e nao bloqueante.
2. Voce precisa baixar 50 paginas web. CPU-bound ou I/O-bound? Qual ferramenta?
3. Voce precisa calcular o hash SHA-256 de 10.000 arquivos. Qual ferramenta e por que o GIL importa aqui?
4. Por que threads nao aceleram um loop de calculo puro em Python?

AULA 2

Yield e Generators

Generators são a base para entender async/await. Uma função com yield vira um generator: pausa no yield, devolve um valor e mantém todo o estado local até ser chamada de novo.

return vs yield

```
def get_products_normal():
    products = Product.objects.all()
    return list(products) # carrega TUDO na memoria

def get_products_lazy():
    for product in Product.objects.all():
        yield product # devolve um de cada vez, sob demanda
```

Chamar `get_products_lazy()` não executa nada: só cria o objeto generator. O código só roda quando você itera (`next()` ou `for`).

```
gen = get_products_lazy()
print(next(gen)) # executa até o primeiro yield
print(next(gen)) # continua de onde parou
```

Por que importa: memoria

Se você tem uma tabela com 2 milhões de linhas e faz `.all()` numa lista, carrega tudo na RAM. Com generator, processa um registro por vez:

```
def process_large_dataset(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip().split(',')

for row in process_large_dataset('logs_2026.csv'):
    processar(row) # nunca carrega o arquivo inteiro
```

Nota

Isso é diretamente útil em relatórios financeiros com milhares de transações: um generator evita estourar a memória do servidor ao gerar o relatório.

Generator expressions

```
squares = (x**2 for x in range(1_000_000)) # generator, lazy
squares_list = [x**2 for x in range(1_000_000)] # lista inteira na memoria
```

Regra prática: se você só vai iterar uma vez (`sum`, `for`), use generator expression. Se precisa de índice ou reuso, use lista.

send(): a ponte para coroutines

O yield também recebe valor de volta — e isso que a Aula 3 usa:


```
def accumulator():
    total = 0
    while True:
        value = yield total
        total += value

acc = accumulator()
next(acc) # 'prime': avanca ate o primeiro yield
print(acc.send(10)) # total = 10
print(acc.send(5)) # total = 15
```

Esse mecanismo de pausar, receber algo e continuar e literalmente o que `async def + await` fazem por baixo dos panos.

Exercicio para fazer a mao

1. Escreva um generator `contar_ate(n)` que produz de 1 ate n usando `yield`.
2. Qual a diferenca de memoria entre `[x for x in range(10**7)]` e `(x for x in range(10**7))`?
3. O que acontece se voce chamar `next()` num generator ja esgotado?
4. Escreva um generator que le um arquivo e produz apenas as linhas que contem a palavra `ERROR`.

AULA 3

Coroutines e o event loop

`async def` cria uma coroutine, que é um generator especializado por baixo dos panos. Chamar a função não executa o corpo — só cria o objeto.

```
async def get_product(product_id):
    return await db.fetch(product_id)

result = get_product(1)
print(result) # <coroutine object ...> (não executou ainda)
```

O que o `await` faz

- Suspende a coroutine atual no ponto do `await`.
- Devolve o controle ao event loop.
- Quando o valor esperado estiver pronto, o event loop retoma a coroutine de onde parou, com as variáveis locais intactas.

```
import asyncio, time

async def fetch_product(id, delay):
    print(f'Buscando produto {id}...')
    await asyncio.sleep(delay) # simula I/O, não trava a thread
    print(f'Produto {id} pronto')
    return id

async def main():
    start = time.time()
    resultados = await asyncio.gather(
        fetch_product(1, 2),
        fetch_product(2, 1),
    )
    print(f'Total: {time.time() - start:.1f}s')

asyncio.run(main())
```

O total é ~2s (o maior delay), não 3s (a soma). As duas coroutines rodam concorrentemente porque `asyncio.sleep` libera o event loop.

O erro clássico: `sleep` errado

```
async def errado():
    time.sleep(2) # BLOQUEIA o event loop inteiro

async def certo():
    await asyncio.sleep(2) # libera o loop para outras coroutines
```

Atenção

Um único `time.sleep()`, uma chamada síncrona de banco ou uma lib HTTP síncrona dentro de uma rota `async` do FastAPI/Django trava o servidor para TODOS os usuários. É o vetor de DoS acidental mais comum em código assíncrono.

Coroutine, Task e event loop

Termo	Papel
Coroutine	A 'receita': objeto de <code>async def</code> , ainda nao agendado.
Task	Coroutine agendada (<code>create_task/gather</code>), rodando concorrentemente.
Event loop	O agendador que decide qual coroutine roda a cada <code>await</code> .

```
async def main():
    task1 = asyncio.create_task(fetch_product(1, 2))
    task2 = asyncio.create_task(fetch_product(2, 1))
    await task1
    await task2
```

Nota

`gather()` cria e aguarda varias coroutines de uma vez — o jeito mais comum quando voce ja sabe todas de antemao. `create_task()` da mais controle: dispara algo e deixa voce continuar antes de esperar o resultado.

Exercicio para fazer a mao

1. Reescreva duas chamadas sequenciais com `await` para rodarem concorrentemente com `gather`.
2. Por que o corpo de uma coroutine nao executa ao ser chamada sem `await`?
3. O que aconteceria com o tempo total se voce trocasse `asyncio.sleep` por `time.sleep` no exemplo?
4. Qual a diferenca entre uma coroutine e uma Task?

AULA 4

CPU parte 1: multiprocessing

Aqui asyncio para de ajudar. Async resolve I/O-bound. Para CPU-bound o GIL trava tudo numa thread — a saída e processos separados, cada um com seu interpretador e seu próprio GIL.

multiprocessing.Process

```
from multiprocessing import Process

def cpu_bound(n):
    return sum(i * i for i in range(n))

if __name__ == '__main__':
    p1 = Process(target=cpu_bound, args=(50_000_000,))
    p2 = Process(target=cpu_bound, args=(50_000_000,))
    p1.start(); p2.start()
    p1.join(); p2.join()
```

Atencao

O `if __name__ == '__main__':` não é estilo — é obrigatório. No Windows o multiprocessing recria o processo importando o módulo do zero; sem essa guarda, você entra em loop infinito de criação de processos.

Recuperando o retorno com Queue

O return de um processo filho não volta sozinho ao pai — a memória é isolada. Para trazer o resultado, usa-se IPC (Queue, Pipe, Value/Array):

```
from multiprocessing import Process, Queue

def cpu_bound(n, queue):
    queue.put(sum(i * i for i in range(n)))

if __name__ == '__main__':
    queue = Queue()
    p1 = Process(target=cpu_bound, args=(10_000_000, queue))
    p1.start(); p1.join()
    print(queue.get())
```

ProcessPoolExecutor: o padrão real

```
from concurrent.futures import ProcessPoolExecutor

def cpu_bound(n):
    return sum(i * i for i in range(n))

if __name__ == '__main__':
    with ProcessPoolExecutor() as executor:
        resultados = list(executor.map(cpu_bound, [50_000_000, 50_000_000]))
    print(resultados) # retorno automatico, sem Queue manual
```

Custo e quando NAO usar

Criar processo tem custo alto: o SO aloca memoria nova e inicializa um interpretador do zero. So compensa para tarefas pesadas o suficiente.

Tarefa	Ferramenta
Calcular hash de milhares de arquivos	multiprocessing
Redimensionar centenas de imagens	multiprocessing
Chamar 100 APIs externas	asyncio
Ler 100 arquivos do disco	asyncio ou threading

Exercicio para fazer a mao

1. Por que o return de um processo filho nao chega ao processo pai sozinho?
2. Reescreva o exemplo de Queue usando ProcessPoolExecutor.
3. Explique por que criar um processo para uma tarefa minima pode ser mais lento que faze-la direto.
4. Cite duas tarefas do seu dia a dia que seriam CPU-bound e se beneficiariam de multiprocessing.

AULA 5

Threads parte 2

Threads não dão paralelismo de CPU (o GIL impede), mas dão concorrência real em I/O: quando uma thread faz I/O, o GIL é liberado e outra thread roda. Isso resolve um caso que async não resolve: bibliotecas síncronas sem versão async.

ThreadPoolExecutor: o padrão de produção

```
from concurrent.futures import ThreadPoolExecutor
import requests

def fetch_url(url):
    return requests.get(url).status_code

urls = ['https://api1.com', 'https://api2.com', 'https://api3.com']

with ThreadPoolExecutor(max_workers=5) as executor:
    resultados = list(executor.map(fetch_url, urls))
```

Nota

Use threads quando a lib disponível é síncrona (requests) e não há opção async (httpx/aiohttp). Se há versão async, asyncio escala melhor.

Race condition: o problema das threads

Threads compartilham memória. Duas threads escrevendo na mesma variável ao mesmo tempo corrompem o dado — mesmo com o GIL, porque ele protege o interpretador, não a lógica da sua aplicação.

```
contador = 0

def incrementar():
    global contador
    for _ in range(1_000_000):
        contador += 1 # NÃO é atômico: ler + somar + escrever

# resultado real fica ABAIXO de 2.000.000 e inconsistente
```

`contador += 1` parece uma operação, mas são três passos (ler, somar, escrever). O GIL pode trocar de thread entre eles, fazendo uma sobrescrever a outra.

Lock: resolvendo a race condition

```
lock = threading.Lock()
contador = 0

def incrementar():
    global contador
    for _ in range(1_000_000):
        with lock: # só uma thread por vez neste bloco
            contador += 1
```

Nota

O Lock deve proteger apenas a secao critica, nunca a funcao inteira — senao voce elimina o ganho de concorrencia. Isso e a versao em memoria do que o `booking_no_overlap` (exclusion constraint) resolve no nivel do banco.

Comparativo final

Criterio	threading	asyncio	multiprocessing
Paralelismo de CPU	Nao	Nao	Sim
Bom para I/O	Sim	Sim (escala +)	Nao
Libs sincronas legadas	Sim	Nao	Sim
Overhead de criacao	Medio	Muito baixo	Alto
Precisa de Lock	Sim	Nao	Nao (mem. isolada)

Exercicio para fazer a mao

1. Por que uma race condition acontece mesmo existindo o GIL?
2. Onde exatamente o Lock deve ser colocado, e por que nao na funcao inteira?
3. Quando voce escolheria threading em vez de asyncio para tarefas de I/O?
4. Relacione o Lock com uma constraint de banco que voce ja usou.

AULA 6

TDD (Test Driven Development)

TDD inverte a ordem: escreve-se o teste antes do código. Isso força você a pensar na interface (o que a função recebe e devolve) antes da implementação, e evita testes que só confirmam o que o código já faz.

O ciclo Red-Green-Refactor

Fase	O que fazer
RED	Escreve um teste que falha (a feature ainda não existe).
GREEN	Escreve o código mínimo para passar (não precisa ser bonito).
REFACTOR	Limpa o código mantendo os testes verdes.

RED: teste que falha primeiro

```
import pytest
from datetime import datetime
from scheduling import is_slot_available

def test_slot_disponivel_sem_conflito():
    assert is_slot_available(datetime(2026,7,10,14,0), []) is True

def test_slot_indisponivel_com_conflito():
    existentes = [datetime(2026,7,10,14,0)]
    assert is_slot_available(datetime(2026,7,10,14,0), existentes) is False
```

GREEN: código mínimo para passar

```
def is_slot_available(horario, existing_bookings):
    return horario not in existing_bookings
```

REFACTOR: só depois de verde

```
from datetime import timedelta

def is_slot_available(horario, existing_bookings, duracao_minutos=30):
    fim_novo = horario + timedelta(minutes=duracao_minutos)
    for existente in existing_bookings:
        fim_existente = existente + timedelta(minutes=duracao_minutos)
        if horario < fim_existente and fim_novo > existente:
            return False
    return True
```

Nota

Se o refactor mudar o comportamento, os testes RED originais acusam na hora. Essa é a garantia central do TDD: refatorar sem medo.

AAA: estrutura de um bom teste

```
def test_slot_indisponivel_com_conflito():
    # Arrange
    existentes = [datetime(2026,7,10,14,0)]
    novo = datetime(2026,7,10,14,0)
    # Act
    resultado = is_slot_available(novo, existentes)
    # Assert
    assert resultado is False
```

Fixtures: eliminando repeticao

```
@pytest.fixture
def horario_base():
    return datetime(2026, 7, 10, 14, 0)

def test_slot_disponivel(horario_base):
    assert is_slot_available(horario_base, []) is True
```

Unitario vs integracao

Tipo	O que valida	Velocidade
Unitario	Uma funcao isolada, sem dependencias externas	Rapido (ms)
Integracao	Fluxo real, com banco/API de verdade	Mais lento

Nota

A mecanica e a mesma no Vitest que voce ja usa no Trimote — muda a sintaxe (pytest no lugar de vitest), nao o habito de test-first com RED visivel.

Exercicio para fazer a mao

1. Escreva o teste RED de uma funcao soma(a, b) antes de implementa-la.
2. Implemente o codigo GREEN minimo para o teste acima passar.
3. Explique por que escrever o teste depois do codigo tende a produzir testes fracos.
4. Escreva 3 casos de teste (incluindo bordas) para uma funcao que valida CPF.

AULA 7

Binary Search

Binary search encontra um elemento em um array ordenado descartando metade do espaço de busca a cada passo. A ordenação previa é a premissa que sustenta o algoritmo.

Implementação iterativa

```
def binary_search(arr, target):
    esquerda, direita = 0, len(arr) - 1
    while esquerda <= direita:
        meio = (esquerda + direita) // 2
        if arr[meio] == target:
            return meio
        elif arr[meio] < target:
            esquerda = meio + 1
        else:
            direita = meio - 1
    return -1
```

Passo a passo — buscando 4 em [1,3,4,7,9,11,13]

```
1a: esq=0 dir=6 meio=3 -> arr[3]=7 (7>4, vai p/ esquerda: dir=2)
2a: esq=0 dir=2 meio=1 -> arr[1]=3 (3<4, vai p/ direita: esq=2)
3a: esq=2 dir=2 meio=2 -> arr[2]=4 (4==4, retorna indice 2)
```

O bug clássico do meio

```
meio = esquerda + (direita - esquerda) // 2 # evita overflow
```

Nota

Em linguagens com inteiro de tamanho fixo (Java, C++), esquerda + direita pode estourar o limite do int. Em Python não há esse problema (int é ilimitado), mas é pergunta clássica de entrevista — vale entender o porque.

Complexidade: aplicando Big O

Algoritmo	Complexidade	Por que
Busca linear	$O(n)$	Pior caso: percorre o array todo
Binary search	$O(\log n)$	Cada passo descarta metade do espaço

Com 1 milhão de elementos: busca linear pode levar 1 milhão de comparações; binary search leva no máximo ~20 ($\log_2$ de 1.000.000 ~ 19,9).

bisect: a versão de produção

```
from bisect import bisect_left

arr = [1, 3, 4, 7, 9, 11, 13]
i = bisect_left(arr, 4)
if i < len(arr) and arr[i] == 4:
    print(f'Encontrado no índice {i}')
```

Nota

Em producao voce usa bisect, nao implementa a mao. bisect_left tambem serve para insercao ordenada — achar onde um valor entra mantendo a ordem.

Atencao

Casos de borda (array vazio, primeiro e ultimo elemento) sao onde binary search mais quebra: esquerda > direita mal tratado causa loop infinito ou indice invalido. Sempre teste esses casos.

Exercicio para fazer a mao

1. Rode o algoritmo a mao buscando o numero 9 em [1,3,4,7,9,11,13]. Anote cada iteracao.
2. O que a funcao retorna ao buscar um numero que nao existe? Por que?
3. Por que binary search nao funciona em um array desordenado?
4. Escreva os testes (RED) para binary_search cobrindo array vazio e elemento ausente.

AULA 8

Bubble Sort

Bubble sort compara pares adjacentes e troca se estiverem fora de ordem. A cada passada, o maior elemento 'borbulha' ate o fim. E didatico, mas raramente usado em producao.

Implementacao

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n - 1):  
        for j in range(n - 1 - i):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
    return arr
```

$n - 1 - i$ reduz o alcance da varredura a cada passada, porque os i maiores elementos ja estao fixos no fim.

Passo a passo — 1a passada de [6,5,3,1,8,7,2,4]

```
[6,5,...] 6>5 troca -> [5,6,3,1,8,7,2,4]  
6>3 troca -> [5,3,6,1,8,7,2,4]  
6>1 troca -> [5,3,1,6,8,7,2,4]  
6>8? nao  
8>7 troca -> [5,3,1,6,7,8,2,4]  
8>2 troca -> [5,3,1,6,7,2,8,4]  
8>4 troca -> [5,3,1,6,7,2,4,8] (8 chegou ao fim)
```

Otimizacao: parada antecipada

```
def bubble_sort_otimizado(arr):  
    n = len(arr)  
    for i in range(n - 1):  
        trocou = False  
        for j in range(n - 1 - i):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
                trocou = True  
        if not trocou:  
            break # ja esta ordenado, para  
    return arr
```

Complexidade

Caso	Complexidade	Cenario
Melhor	$O(n)$	Array ja ordenado (com otimizacao)
Medio	$O(n^2)$	Ordem aleatoria
Pior	$O(n^2)$	Ordem inversa

Atencao

Ninguem usa bubble sort em producao. Python ja tem `sorted()` e `.sort()`, implementados com Timsort ($O(n \log n)$ garantido, otimizado em C). Bubble sort existe por valor didatico — em codigo real, sempre `sorted()`.

```
arr = [6, 5, 3, 1, 8, 7, 2, 4]
print(sorted(arr)) # [1,2,3,4,5,6,7,8] – muito mais rapido
```

Exercicio para fazer a mao

1. Complete a mao a 2a passada do bubble sort sobre [5,3,1,6,7,2,4,8].
2. O que a flag 'trocou' economiza, e em qual cenario ela mais ajuda?
3. Por que o alcance do loop interno diminui com $n - 1 - i$?
4. Compare a curva $O(n^2)$ do bubble com $O(\log n)$ do binary search para $n = 1.000.000$.

AULA 9

Ferramentas, sites e comunidades

Pratica constante e o que consolida a teoria. Abaixo, ferramentas atuais organizadas por objetivo, para montar uma rotina de treino.

Pratica de algoritmos e logica

- LeetCode — referencia em algoritmos e estruturas de dados, do iniciante ao avancado; foco em preparacao para entrevistas.
- HackerRank — desafios em varias categorias (algoritmos, matematica, IA), com avaliacao automatica do codigo.
- Codewars — pratica gamificada via 'kata' que aumentam de dificuldade; otimo para criar habito diario.
- Exercism — open source, com mentoria humana revisando seu codigo.
- Beecrowd (ex-URI) — brasileiro, +3000 problemas com enunciado em portugues; bom para fixar logica sem barreira de idioma.
- Codeforces — programacao competitiva, nivel avancado; proximo passo quando o LeetCode ficar confortavel.

Foco em Python

- awesome-python (github.com/vinta/awesome-python) — lista curada de bibliotecas por categoria.
- Python Fluente (pythonfluente.com) — referencia em Python idiomático.
- Real Python (realpython.com) — tutoriais aprofundados.
- PyBites / Python Morsels — exercicios curtos focados em escrever Python idiomático, nao so logica generica.

Comunidades e foruns (BR)

- TabNews — 'Hacker News brasileiro' de Filipe Deschamps; referencia de discussao tecnica em portugues.
- DIO — bootcamps gratuitos e Discord ativo.
- Discord Python Brasil — servidor focado em Python em portugues.
- Discord da Rocketseat — categorias por linguagem, banco, agil e IA.

Preparacao para entrevista

- CodeSignal — exercicios no formato usado em processos seletivos.
- Codility — plataforma que as proprias empresas usam para testes tecnicos.

Nota

Muitos Tech Leads preferem contratar quem e ativo em comunidade ou open source a analisar curriculos frios. Isso reforca a estrategia de 'build in public' — mostrar o que voce constroi vale como portfolio.

Montando a rotina

Objetivo	Ferramenta
Fixar logica todo dia, sem pressao	Codewars, Beecrowd
Algoritmos + estrutura de dados serio	LeetCode, Codeforces
Python idiomatico	PyBites, Python Morsels
Tirar duvida / rede de contatos BR	TabNews, Discord PyBR, DIO
Preparar para processo seletivo	CodeSignal, Codility

Exercicio para fazer a mao

1. Escolha 1 plataforma de pratica e resolva 1 desafio facil por dia nesta semana.
2. Crie conta no TabNews e leia 3 discussoes tecnicas sobre um tema que voce estuda.
3. Implemente do zero (sem consultar) o `binary_search` e o `bubble_sort` desta apostila.

EXERCICIO FINAL

Web scraping assincrono

Objetivo: implementar um web scraping usando Python e asyncio para analisar os dados de uma pagina web. Portamos a versao original (que usava ThreadPoolExecutor + requests) para asyncio + aiohttp.

Decisoões de projeto

- Escrita do CSV movida para o final, depois de todas as coroutines terminarem — elimina a race condition da versao original (cada thread abria o arquivo em modo append e escrevia direto, sem lock).
- Checagem de duplicidade por titulo antes de gravar, ausente na versao original.

Nota

Dependencia nova: pip install aiohttp — cliente HTTP assincrono que substitui o requests (sincrono, que bloqueia a thread durante a espera de rede).

Parte A — o codigo completo

```
import asyncio
import csv
import time
from typing import Optional

import aiohttp
from bs4 import BeautifulSoup

BASE_URL = 'https://havokkморands.github.io/movie-catalog/'
OUTPUT_FILE = 'movies_async.csv'
MAX_CONCURRENT_REQUESTS = 10
REQUEST_TIMEOUT = aiohttp.ClientTimeout(total=20)

HEADERS = {
    'User-Agent': 'Mozilla/5.0 ... Chrome/124.0.0.0 Safari/537.36',
    'Accept-Language': 'en-US,en;q=0.9',
}
```

```
async def fetch_html(session, url):
    try:
        async with session.get(url, headers=HEADERS,
                               timeout=REQUEST_TIMEOUT) as response:
            response.raise_for_status()
        return await response.text()
    except (aiohttp.ClientError, asyncio.TimeoutError) as error:
        print(f'Falha ao acessar {url}: {error}')
    return None
```



```
def extract_movie_links(html):
    soup = BeautifulSoup(html, 'html.parser')
    container = soup.find('section', attrs={'data-testid': 'movies-list'})
    if container is None:
        return []
    items = container.find_all('article', attrs={'data-testid': 'movie-item'})
    return [
        'https://havokkmorands.github.io/' + a['href']
        for item in items
        if (a := item.find('a', attrs={'data-testid': 'movie-link'}, href=True))
    ]
```

```
def parse_movie_details(html, movie_link):
    soup = BeautifulSoup(html, 'html.parser')
    detail = soup.find('section', attrs={'data-testid': 'movie-detail'})
    if detail is None:
        return None

    def get_field(testid, prefix=''):
        tag = detail.find(attrs={'data-testid': testid})
        return tag.get_text(strip=True).replace(prefix, '').strip() if tag else None

    movie = {
        'title': get_field('movie-title'),
        'date': get_field('movie-release', 'Lancamento:'),
        'rating': get_field('movie-rating', 'Nota:'),
        'plot': get_field('movie-synopsis', 'Sinopse:'),
    }
    return movie if all(movie.values()) else None
```

```
async def fetch_movie(session, semaphore, movie_link):
    async with semaphore:
        html = await fetch_html(session, movie_link)
        return parse_movie_details(html, movie_link) if html else None
```

```
def save_movies(movies):
    seen_titles = set()
    unique = []
    for movie in movies:
        if movie['title'] in seen_titles:
            continue
        seen_titles.add(movie['title'])
        unique.append(movie)

    with open(OUTPUT_FILE, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(['title', 'date', 'rating', 'plot'])
        for m in unique:
            writer.writerow([m['title'], m['date'], m['rating'], m['plot']])
```

```
async def main():
    semaphore = asyncio.Semaphore(MAX_CONCURRENT_REQUESTS)
    async with aiohttp.ClientSession() as session:
        index_html = await fetch_html(session, BASE_URL)
        if index_html is None:
            return
        movie_links = extract_movie_links(index_html)
        tasks = [fetch_movie(session, semaphore, link) for link in movie_links]
        results = await asyncio.gather(*tasks)
        movies = [m for m in results if m is not None]
        save_movies(movies)

if __name__ == '__main__':
    asyncio.run(main())
```

Parte B — explicacao linha por linha

Imports

Import	Papel
asyncio	Fornece Semaphore, gather, run e o event loop.
csv	Escreve CSV tratando virgulas/aspas dentro dos campos.
time	Mede o tempo de execucao do script.
Optional	Type hint: 'ou str, ou None'. Documenta a intencao.
aiohttp	Cliente HTTP assincrono; substituto do requests.
BeautifulSoup	Parser de HTML; nao muda entre versao sync e async.

Nota

BeautifulSoup nao e async porque parsing de HTML e processamento local, nao I/O — nao ha o que aguardar.

Constantes de configuracao

- `MAX_CONCURRENT_REQUESTS = 10` — limita quantas requisicoes HTTP ficam abertas ao mesmo tempo. E o equivalente ao `MAX_THREADS` da versao original.
- `REQUEST_TIMEOUT` — no aiohttp o timeout e um objeto (`ClientTimeout`), que permite granularidade. `total=20` e o timeout global da operacao.

Headers

O User-Agent faz a requisicao parecer vinda de um navegador. Sem ele, muitos servidores identificam o User-Agent padrao de bibliotecas (ex: python-requests) e bloqueiam. O Accept-Language fixa o idioma do conteudo, evitando resultado inconsistente entre execucoes.

fetch_html — a requisicao

- `async def` — declara coroutine; chamar nao executa, so cria o objeto.
- `session` como parametro — reusa o pool de conexoes TCP; criar sessao nova a cada chamada desperdicaria esse reaproveitamento.

- `async with session.get(...)` — versao assincrona do `with`; garante o fechamento da conexao mesmo em erro, e o `await` implicito pausa a coroutine esperando a resposta.
- `raise_for_status()` — levanta excecao em erro HTTP (404, 500); sem isso, uma pagina de erro seria tratada como valida.
- `await response.text()` — baixar o corpo tambem e I/O, entao precisa de `await`.
- `except` especifico — captura `ClientError` e `TimeoutError`, nao `Exception` generico, para nao esconder bugs do proprio codigo.
- `return None` — em falha, devolve `None` em vez de derrubar tudo; assim a falha de um filme nao interrompe os outros.

extract_movie_links — parse da pagina indice

- Nao e `async`: so processa uma string ja baixada.
- `data-testid` sao atributos que o site expoe para automacao/testes (padrao de Cypress/Playwright), reaproveitados aqui para scraping.
- Guard clause (`if container is None`) aborta cedo em vez de quebrar adiante com `AttributeError`.
- Walrus (`:=`) na list comprehension atribui e testa na mesma expressao: so inclui o link se a tag `<a>` existir e tiver `href`.
- O `href` e relativo, entao e concatenado com o dominio para virar URL completa.

parse_movie_details — dados de cada filme

- `movie_link` serve so para log de erro, nao para logica.
- Funcao aninhada `get_field` — extrai uma vez o padrao 'busca tag -> pega texto -> remove prefixo -> tira espaco', reusado nos 4 campos.
- `get_text(strip=True)` pega so o texto visivel, sem tags internas.
- `all(movie.values())` — so devolve o filme se TODOS os 4 campos foram extraidos; se faltou um, descarta o registro (validacao de integridade).

fetch_movie — controle de concorrencia

- Semaphore e um contador que limita quantas coroutines entram no bloco ao mesmo tempo. Com `Semaphore(10)`, a 11a fica pausada esperando vaga.
- `async with semaphore` consome uma vaga ao entrar e devolve ao sair (mesmo em erro). Faz o papel do `MAX_THREADS`.
- O semaforo esta DENTRO desta funcao para limitar requisicoes ABERTAS, nao tarefas criadas: as 50 tarefas sao criadas de uma vez, mas so 10 fazem rede ao mesmo tempo.

save_movies — duplicidade + escrita

- `set()` para `seen_titles`: checar 'in set' e $O(1)$; em lista seria $O(n)$. E o habito certo — estrutura de dados adequada a operacao dominante.
- Este e o ponto que resolve a race condition: a escrita so ocorre depois que todas as coroutines terminaram, num loop sequencial simples.
- `mode='w'` sobrescreve o arquivo (o 'a' original acumulava e duplicava entre execucoes).
- `newline=''` — exigencia do modulo `csv`; sem isso, no Windows sairiam linhas em branco extras.

- `encoding='utf-8'` — garante acentos corretos independente do SO.
- `writer.writerow` trata virgulas/aspas dentro dos campos automaticamente.

main — orquestracao

- Semaphore criado uma vez e passado a todas as tarefas: precisa ser o MESMO objeto, senao nao ha limite real.
- `ClientSession` criada uma vez, reusada por todas as requisicoes, fechada ao sair do `async with`.
- `tasks = [...]` — a list comprehension NAO executa nada: so cria os objetos coroutine.
- `await asyncio.gather(*tasks)` — aqui tudo comeca a rodar concorrentemente; results volta na MESMA ordem das tarefas.
- `[m for m in results if m is not None]` — descarta os None (falhas) antes de salvar.

Bloco de entrada

if `__name__ == '__main__'` so roda ao executar o arquivo diretamente. `asyncio.run(main())` cria o event loop, roda `main()` ate o fim e fecha o loop — e o que 'liga a energia' de todas as coroutines.

O que muda vs a versao em threading

Threading (original)	Async (esta versao)
<code>requests.get()</code> bloqueia a thread	<code>session.get()</code> com <code>await</code> libera o loop
<code>MAX_THREADS + executor.map()</code>	<code>Semaphore(10)</code> dentro do <code>gather()</code>
Cada thread escreve direto (append)	CSV escrito uma vez, no final
Sem checagem de duplicidade	<code>seen_titles</code> filtra antes de gravar

Atencao

O `parse_movie_details` roda `BeautifulSoup` dentro da coroutine — parsing e CPU-bound e bloqueia o event loop enquanto executa. Para este volume o impacto e irrelevante (o gargalo e a rede). Se escalar para milhares de paginas com HTML pesado, mova o parsing para `loop.run_in_executor()`.

Exercicio para fazer a mao

1. Rode o script e confira o `movies_async.csv` gerado. Quantos filmes vieram?
2. Mude `MAX_CONCURRENT_REQUESTS` para 1 e para 50. O tempo total muda? Por que?
3. Explique com suas palavras por que a versao async nao tem race condition na escrita.
4. Sugira a mensagem de commit (Conventional Commits) para adicionar este script ao repo.